

In-app messaging — findings from a production integration

Context: Teamder (React Native, Android), Joryio Android SDK vendored from HippoCampus-Tech/Hippomation . Workspace 33b93794-3468-4241-aeaf-d752166ae066 , campaign 9a077b7e-a2a4-48cd-8725-cc3102531ca5 .

The campaign rendered correctly on a real device and reported **impressions: 3, clicks: 0, dismissals: 0** — after a session in which the CTA was tapped and the message was closed. What follows is the trace to the cause, then the issues that are yours.

There were **two** causes, one in each codebase, and they masked each other. Ours is written up first (§0) because it had to be fixed before yours was even visible. Yours (§1) is the one that kept clicks at zero after that, and it is a bug you have already fixed upstream but which no released SDK carries — so every integrator on the current published sources hits it.

Both are the same *shape* of defect, which is the point worth taking from this report: an interaction that is not recorded still answers `{"success": true}` , and the SDK still logs "Impression tracked successfully".

0. What the cause turned out to be (ours) — but why the API invited it

`trackInAppImpression(campaignId, action)` takes `action: String` . The backend classifies it by **substring**:

```
// InAppMessagingManager.kt
action = when {
    normalized.contains("click")    -> "click"
    normalized.contains("dismiss") -> "dismiss"
    else -> action
},
clicked      = if (normalized.contains("click"))    true  else null,
dismissedAt = if (normalized.contains("dismiss")) nowIso else null,
```

and the server then treats "no interaction marker" as a display:

```
// in-app-evaluation.service.ts
const reportsDisplay =
  !!displayedAt || (!clicked && !dismissedAt && !actionIsInteraction && !converted);
```

We were passing `'impression'` for a display and `'button:cta'` for a CTA tap. Neither contains `click` or `dismiss`, so **both were recorded as displays**. Hence 3 impressions (one real display + two button taps) and 0 clicks. Every layer underneath — SDK, transport, delivery token, ClickHouse aggregation — was working correctly the entire time.

Verified: replaying the exact wire payloads by hand against `POST /v1/in-app/track` with a valid delivery token produced `clicks: 1, dismissals: 1, ctr: 25` within a minute. The pipeline is sound.

Issue 0 — the action parameter should not be a free `String`

This is a silent-failure API: every wrong value returns `{"success": true}`, is stored, and shows up as a *plausible* number. Nothing in the SDK, the response, or the dashboard distinguishes "you reported a click" from "you reported a third display". We lost a working feature to it for a day, and the symptom (impressions inflated, clicks zero) actively pointed away from the cause.

Suggested fixes, in order of preference:

1. **Type it.** `enum class InAppAction { DISPLAYED, CLICKED, DISMISSED, CONVERTED }` on Android/iOS, a union on the RN and web SDKs. The mistake stops compiling.
2. If the `String` overload must stay for compatibility, **`**reject unknown values loudly**`** — `logger.warn("Unknown in-app action '$action'; expected one of ...")` — rather than silently classifying them as a display.
3. Make the server honest too: `reportsDisplay` currently treats *any* unrecognised action as a display. An action string that matches nothing known is a client bug, and `{ success: true, recorded: false, reason: 'unknown_action' }` would say so — the same honesty the bot-filter path already applies, and for the same reason documented there.

Note that substring matching is itself fragile in a way worth removing: an action of `"dismissed-after-click"` classifies as a click, and `"unclicked"` classifies as a click too.

1. The shipped Android SDK cannot report a click at all

Severity: critical — the in-app channel reports zero engagement on Android.

`InAppMessagingManager.trackImpression` in the sources we vendored builds this:

```

val request = TrackImpressionRequest(
    campaignId = campaignId,
    userId = identityManager.getUserId() ?: "",
    anonymousId = identityManager.getAnonymousId(),
    sessionId = sessionManager.getSessionId(),
    action = action,           // ← the interaction, and nothing else
    deliveryToken = deliveryToken
)

```

Captured off the wire from a release build with body logging on:

```

--> POST /v1/in-app/track
{"action":"dismissed","anonymousId":"","campaignId":"","deliveryToken":""}
<-- 200
{"success":true}

```

No `in_app.dismissed` event was written. The backend classifies an impression from the STRUCTURED markers — `displayedAt` / `clicked` + `clickedAt` / `dismissedAt` — and reads "no marker present" as a display:

```

const actionIsInteraction =
    action.includes('click') || action.includes('dismiss') || action.includes('convert')
const reportsDisplay =
    !!displayedAt || (!clicked && !dismissedAt && !actionIsInteraction && !converted);
...
if (impressionData.clicked) { emit('in_app.clicked', ...) }

```

So `action: "clicked"` with no markers falls between the branches: not a display (the string contains `click`, so `actionIsInteraction` is true), and not a click (`clicked` is undefined). **Nothing is recorded, and the response is 200 `{"success": true}`**. Only `displayed` works, because it is the one action that happens to land in the fallback.

Replaying the identical request by hand with the markers added recorded a click within the minute — that is how the client half was isolated from the server half.

Your current `main` already fixes this: the newer `trackImpression` maps the action onto the markers and collapses it to the singular `click` / `dismiss` the web SDK sends. We fixed our integration by re-vendoring from `main`. But the published artifacts still carry the old shape, so **anyone integrating from the released SDK today gets an in-app channel that silently reports 0% CTR**. That is worth a release rather than waiting for the next one.

1a. The SDK reads the HTTP status and ignores the body

```
when (val result = networkClient.trackImpression(request)) {  
    is NetworkResult.Success -> logger.debug("Impression tracked successfully")  
}
```

`NetworkResult.Success` means HTTP 2xx. But `/v1/in-app/track` answers **200** with `{"success": false, "error": ...}` for every rejection it knows about — invalid delivery token, missing token under `IN_APP_REQUIRE_DELIVERY_TOKEN`, tenant mismatch. The SDK logs "succeeded" for all of them.

Combined with §1b below, a rejected impression is indistinguishable from one that was never sent, from inside the app *and* from the logs.

1b. Every log line is gated behind `enableDebug`

`Logger.error` is gated on the same `enabled` flag as `verbose`. An integration that has not opted into debug logging gets **complete silence** in logcat — no warning, no error, nothing — while impressions are being dropped. Errors are not debug output; they should print regardless of level, exactly so that this class of failure announces itself.

2. `push_permission` is stamped once and never refreshed

Severity: high — it silently shrinks every push-targeted audience.

A user who grants notification permission *after* the SDK's first init keeps `push_permission = not_determined` in Joryio forever. Confirmed on a fresh install: permission granted, device row created, `pushToken` stored correctly, and the attribute still read `not_determined`.

That is the common path, not an edge case — the OS prompt is deliberately deferred until after sign-up in most apps (Apple's 5.1.1 friction guidance), so init *always* runs first. Any segment filtering on `push_permission = granted` therefore excludes real, reachable users.

`registerPushToken(token)` is the natural place to fix it: reaching it means the OS returned a token, which means permission was granted. We work around it client-side today by calling `getPushPermissionStatus()` and writing the attribute back ourselves after every registration.

3. Two device rows are created for one device

For a single install, `GET /users/{id}/devices` returns two rows:

<code>_ID</code>	<code>DEVICEID</code>	<code>PUSHTOKEN</code>
<code>6a8a0de6...</code>	<code>584bfe30-...</code>	<i>(absent)</i>
<code>6a8a0de3...</code>	<i>(absent)</i>	<code>chaLXmtTQ4ms...</code>

One row carries the device identity, the other carries the token, and nothing joins them. Created three seconds apart on first launch, so it looks like the init/device-registration path and the push-registration path each create a row against different keys instead of upserting on one.

Consequences: device counts are doubled, and any logic that reads "the device" gets a 50/50 chance of the row without the token.

4. `Joryio.kt` discards `present()`'s return value, contradicting its own comment

```
// Joryio.kt:~893
inAppMessagingManager.noteDisplayed()
// Only track when the presenter says it actually drew.
// Counting an impression for a message that failed to
// display is how a channel looks healthier than it is.
presenter.present(activity, campaign) { action, _ ->
    trackInAppImpression(campaign.id, action)
}
```

`present()` returns `Boolean`, and both `DefaultInAppMessagePresenter` and `InAppMessageRenderer.show()` are carefully written to return `false` when they decline to draw (null content, `type == CUSTOM`, HTML without opt-in, or a `Throwable`). Each of those paths carries a comment explaining that returning `false` is what stops a phantom impression being counted.

The return value is then dropped at the call site. Worse, `noteDisplayed()` is called *before* `present()`, so a message that never drew still spends the cross-campaign frequency budget — the exact outcome the `activity == null` branch immediately above goes out of its way to avoid.

The three comments describe behaviour the code does not have.

5. Minor: two packaging gaps that cost a build each

Not bugs in the product, but each one produced a *successful* build that did nothing, which is expensive to diagnose:

- `packages/sdk-react-native` ships no `react-native.config.js`. Without it, RN autolinking silently skips the package: the app compiles, the JS module resolves, and the APK contains zero SDK classes. We add the file locally after every re-vendor. Shipping it in the package would remove the failure mode.
- `joryio-react-native.podspec` pins `s.dependency "Joryio", "~> 1.0"`, which is not published to any spec repo, so `pod install` cannot resolve it. We repoint `s.source` and drop the version pin locally.

What we verified working, for the record

- `POST /v1/track` — events land, `type` is the correct field (not `name`).
- `POST /v1/in-app/sync` — targeting is correctly isolated: the probe user gets the campaign, a real user gets zero.
- Delivery tokens — minted per sync, `IN_APP_REQUIRE_DELIVERY_TOKEN` is on and correctly rejects tokenless calls.
- `POST /v1/in-app/track` — displayed / clicked / dismissed all record correctly and aggregate into `stats` and `ctr` within about a minute.
- `POST /v1/push/register` — the token stored in Joryio matches the FCM token in our own backend byte for byte.
- Bot filtering — worth flagging for other integrators: React Native's default `User-Agent: okhttp/4.12.0` classifies as `other_bot`, and every analytics query filters `$is_bot != 'true'`. The data is ingested and then invisible. The `recorded: false, reason: 'bot_filtered'` response is a genuinely good piece of design; the SDKs should set a real UA so it rarely triggers.